SZTUCZNA INTELIGENCJA
I
INŻYNIERIA WIEDZY

•

LISTA 2

Zofia Różańska, 280526

6 MAJA 2026

Spis treści

1 Wstęp **2**

2 Krótki opis zastosowanych algorytmów **2**

2.1 Minimax 2

2.2 Alfa-beta cięcie 2

3 Reprezentacja gry **3**

4 Implementacja **3**

4.1 config.py 4

4.2 board.py 4

4.3 game.py 5

4.4 heuristics.py 6

4.5 algorithms.py 6

4.6 engine.py 9

4.7 main.py 10

5 Testy i obserwacje **11**

5.1 Głębokości i heurystyki 11

5.2 Minimax vs alfa-beta 12

5.3 Rozmiar planszy 12

5.4 Heurystyki 13

5.5 Wnioski 13

6 Podsumowanie **13**

7 Materiały źródłowe **13**

1. Wstęp

Celem projektu było zaimplementowanie gry *Breakthrough* wraz z algorytmami *Minimax* oraz *Minimax z cięciami alfa-beta*. Program został napisany w Pythonie i uruchamia się go za pomocą CLI (command line interface).

Breakthrough to dwuosobowa, deterministyczna gra planszowa o sumie zerowej. Każdy gracz stara się przeprowadzić jeden ze swoich pionków na przeciwny koniec planszy – wygrywa ten, który dokona tego jako pierwszy.

Do wyboru ruchów graczy wykorzystujemy algorytm Minimax oraz jego modyfikację z cięciami alfa-beta. Dodatkowo, program korzysta z prostych heurystyk oceniających aktualny stan planszy.

2. Krótki opis zastosowanych algorytmów

2.1. Minimax

Minimax przeszukuje drzewo możliwych ruchów. Algorytm zakłada, że obaj gracze grają optymalnie:

- gracz MAX wybiera ruch dający najlepszy wynik,
- gracz MIN wybiera ruch dający najgorszy wynik.

Ze względu na to, że liczba wierzchołków wzrasta wykładniczo razem z głębokością, w praktyce stosujemy limit głębokości. Po osiągnięciu limitu, stan „niedokończonej” planszy oceniany jest za pomocą wybranej heurystyki.

2.2. Alfa-beta cięcie

Alfa-beta cięcie jest optymalizacją algorytmu Minimax. Pozwalają pomijać fragmenty drzewa, które i tak już nie wpłyną na końcową decyzję. Dzięki temu algorytm odwiedza znacznie mniej węzłów i działa szybciej, zachowując taki sam wynik jak zwykły Minimax.

3. Reprezentacja gry

Stan planszy przechowywany jest jako tablica ciągów znaków zawierająca w każdej komórce jeden z literałów:

- `W` – pionek gracza 1,
- `B` – pionek gracza 2,
- `_` – puste pole.

Przykładowy stan planszy (6×6):

```
B B B B B B
B W B _ B _
_ _ _ B _ B
_ W _ _ _ _
_ _ W W W W
W W W W W W
```

Gracz **W** zaczyna na dole planszy i porusza się do góry. Gracz **B** zaczyna u góry i porusza się w dół. Gracze poruszają się naprzemiennie.

W każdej turze gracz ma do wyboru ruch jednym ze swoich pionków. Gracz może:

- przesunąć pionek o jedno pole do przodu prosto lub do przodu po skosie, jeśli pole jest puste,
- zbić pionek przeciwnika i zająć jego miejsce, jeśli znajduje się on po skosie.

Gra kończy się, gdy pionek dotrze do przeciwnego końca planszy.

4. Implementacja

Projekt podzieliłam na 7 modułów:

Plik	**Opis**
`config.py`	podstawowa konfiguracja gry
`board.py`	tworzenie i wyświetlanie planszy
`game.py`	generowanie i aplikowanie ruchów, sprawdzanie końca gry
`heuristics.py`	6 funkcji heurystycznych
`algorithms.py`	implementacja algorytów: Minimax i Alfa-beta cięcie
`engine.py`	główna pętla rozgrywki
`main.py`	uruchamianie programu z CLI

4.1. config.py

Plik konfiguracyjny zawiera:

- zdefiniowane literały dla każdego możliwego stanu pola planszy,
- wynik wygranej oraz przegranej (używane przy algorytmach przeszukiwania drzewa gry do rozróżnienia zakończenia rozgrywki),
- identyfikatory algorytmów,
- funkcję zwracającą przeciwnika podanego gracza.

4.2. board.py

Plansza jest reprezentowana przez tablicę ciągów znaków, a punkt (0, 0) znajduje się w lewym górnym rogu planszy, więc:

- gracz W zaczyna w rzędach $rows - 1$ i $rows - 2$ i dąży do rzędu 0,
- gracz B zaczyna w rzędach 0 i 1 i dąży do rzędu $rows - 1$.

Listing 1: Funkcja tworząca początkową planszę

```
def make_initial_board(rows: int = 8, cols: int = 8) -> list[list[str]]:
    board = [[EMPTY] * cols for _ in range(rows)]

    # black on top
    for row in range(2):
        for col in range(cols):
            board[row][col] = PLAYER_B

    # white on bottom
    for row in range(rows - 2, rows):
        for col in range(cols):
            board[row][col] = PLAYER_W

    return board
```

4.3. game.py

Moduł `game.py` zawiera podstawową logikę gry w Breakthrough:

- funkcję generującą możliwe ruchy dla podanego gracza na podanej planszy: `get_moves()`
- funkcję generującą kopię planszy po wykonanym ruchu: `apply_move()`
- funkcję sprawdzającą, czy któryś z graczy wygrywa. Aby uniknąć błędów, dodałam warunek, według którego gracz, który nie ma żadnych dostępnych ruchów, również przegrywa.

Aby wygenerować wszystkie możliwe ruchy dla gracza, dla każdego jego pionka sprawdzamy wszystkie trzy dostępne typy ruchów zdefiniowane w zasadach gry. Odzwierciedla to poniższy kod:

Listing 2: Funkcja generująca możliwe ruchy

```
def get_moves(board: list[list[str]], player: str) -> list[Move]:
    rows = len(board)
    cols = len(board[0])
    direction = -1 if player == PLAYER_W else 1
    opponent = PLAYER_B if player == PLAYER_W else PLAYER_W
    moves: list[Move] = []

    for row in range(rows):
        for col in range(cols):
            if board[row][col] != player:
                continue
            next_row = row + direction
            if not (0 <= next_row < rows):
                continue

            # step straight forward onto an empty cell (no capture)
            if board[next_row][col] == EMPTY:
                moves.append(((row, col), (next_row, col)))

            # step diagonally forward onto an empty cell (capture or no capture)
            for dir_col in (-1, 1):
                next_col = col + dir_col
                if 0 <= next_col < cols and board[next_row][next_col] in (EMPTY,
     opponent):
                    moves.append(((row, col), (next_row, next_col)))

    return moves
```

4.4. heuristics.py

Zaimplementowałam sześć prostych heurystyk. Każda z nich posiada sygnaturę $(board,\ player) \rightarrow float$

Heurystyka	Opis
`heuristic_piece_count`	różnica liczby pionków graczy pozostałych na planszy
`heuristic_advancement`	suma stopni zaawansowania pionków w dążeniu do celu
`heuristic_mobility`	liczba legalnych ruchów do wykonania
`heuristic_capture_potential`	liczba pionków przeciwnika, które gracz może zbić w następnym ruchu
`heuristic_passed_pieces`	liczba pionków gracza, które mają wolną drogę do wygranej
`heuristic_combined`	ważona kombinacja wszystkich powyższych statystyk

Przed każdym ruchem heurystyka jest wybierana losowo, osobno dla każdego gracza.

4.5. algorithms.py

W module `algorithms.py` znajduje się funkcja `choose_move()`, która jest punktem wejścia do algorytmów przeszukiwania drzewa gry: *Minimax* oraz *Alfa-beta cięcia*.

Poniżej przedstawiona jest implementacja algorytmów.

Funkcja `minimax_plain()` implementuje klasyczny algorytm Minimax bez optymalizacji. Rekurencyjnie przeszukuje kolejne poziomy drzewa gry, aż do osiągnięcia stanu końcowego lub limitu głębokości. Parametr `maximizing` określa, czy aktualny poziom drzewa należy do gracza maksymalizującego czy minimalizującego wynik. Po osiągnięciu warunku stopu wykonywana jest ocena heurystyczna planszy.

Listing 3: Algorytm Minimax

```
def minimax_plain(
    board: list[list[str]],
    depth: int,
    maximizing: bool,
    current_player: str,
    root_player: str,
    heuristic_id: int,
) -> tuple[float, tuple | None]:
    global nodes_visited
    nodes_visited += 1

    # Terminal state
    winner = check_winner(board)
    if winner is not None:
        return (WIN_SCORE + depth if winner == root_player else LOSS_SCORE -
    depth), None
```

```
    # Depth cutoff - apply heuristic evaluation
    if depth == 0:
        return evaluate(board, root_player, heuristic_id), None

    moves = get_moves(board, current_player)
    if not moves:
        val = LOSS_SCORE if current_player == root_player else WIN_SCORE
        return val, None

    opp = opponent(current_player)
    best_move = moves[0]

    if maximizing:
        best_val = float("-inf")
        for move in moves:
            val, _ = minimax_plain(
                apply_move(board, move), depth - 1, False, opp, root_player,
    heuristic_id
            )
            if val > best_val:
                best_val, best_move = val, move
        return best_val, best_move
    else:
        best_val = float("inf")
        for move in moves:
            val, _ = minimax_plain(
                apply_move(board, move), depth - 1, True, opp, root_player,
    heuristic_id
            )
            if val < best_val:
                best_val, best_move = val, move
        return best_val, best_move
```

Druga implementacja wykorzystuje alfa-beta cięcia. Działanie algorytmu jest takie samo jak w przypadku zwykłego Minimax, jednak część gałęzi drzewa może zostać pominięta, jeśli wiadomo, że nie wpłyną już na końcową decyzję. Parametry `alpha` i `beta` przechowują aktualnie najlepsze znalezione wartości dla obu graczy i pozwalają na wykonywanie cięć drzewa.

Listing 4: Algorytm Alfa-beta cięcia

```
def minimax_alphabeta(
    board: list[list[str]],
    depth: int,
    maximizing: bool,
    current_player: str,
    root_player: str,
    heuristic_id: int,
    alpha: float,
    beta: float,
```

```
) -> tuple[float, tuple | None]:
    global nodes_visited
    nodes_visited += 1

    # Terminal state
    winner = check_winner(board)
    if winner is not None:
        return (WIN_SCORE + depth if winner == root_player else LOSS_SCORE -
    depth), None

    # Depth cutoff
    if depth == 0:
        return evaluate(board, root_player, heuristic_id), None

    moves = get_moves(board, current_player)
    if not moves:
        val = LOSS_SCORE if current_player == root_player else WIN_SCORE
        return val, None

    opp = opponent(current_player)
    best_move = moves[0]

    if maximizing:
        best_val = float("-inf")
        for move in moves:
            val, _ = minimax_alphabeta(
                apply_move(board, move), depth - 1, False,
                opp, root_player, heuristic_id, alpha, beta,
            )
            if val > best_val:
                best_val, best_move = val, move
            alpha = max(alpha, best_val)
            if alpha >= beta:   # beta cut-off
                break
        return best_val, best_move
    else:
        best_val = float("inf")
        for move in moves:
            val, _ = minimax_alphabeta(
                apply_move(board, move), depth - 1, True,
                opp, root_player, heuristic_id, alpha, beta,
            )
            if val < best_val:
                best_val, best_move = val, move
            beta = min(beta, best_val)
            if alpha >= beta:   # alpha cut-off
                break
        return best_val, best_move
```

Obie funkcje zwracają ocenę aktualnego stanu oraz najlepszy znaleziony ruch. Do-

datkowo, podczas działania zliczana jest liczba odwiedzonych węzłów, co pozwala porównywać wydajność obu algorytmów.

W stanach końcowych do wartości wygranej lub przegranej dodawana jest także aktualna głębokość przeszukiwania. Dzięki temu algorytm preferuje szybsze zwycięstwa i jednocześnie stara się jak najdłużej opóźniać przegraną.

4.6. engine.py

Moduł `engine.py` odpowiada za przebieg całej rozgrywki. Znajduje się w nim funkcja `play_game()`, która steruje kolejnymi turami graczy i komunikuje się z modułami odpowiedzialnymi za algorytmy, heurystyki oraz obsługę planszy.

Funkcja przyjmuje jako argumenty:

- początkową planszę,
- głębokość przeszukiwania,
- wybrany algorytm (`minimax` lub `alphabeta`),
- maksymalną liczbę rund,
- tryb szczegółowego wypisywania ruchów.

Główna część funkcji działa w pętli wykonującej kolejne tury. W każdej rundzie:

1. sprawdzany jest warunek zakończenia gry,
2. losowana jest heurystyka dla aktualnego ruchu,
3. wywoływana jest funkcja `choose_move()` (punkt wejścia dla algorytmów przeszukiwania drzewa gry),
4. wykonywany jest wybrany ruch,
5. aktualizowana jest plansza i liczba rund.

Aby usprawnić działanie programu, wprowadziłam kilka modyfikacji do bazowej wersji. Jeżeli gracz nie ma legalnych ruchów, automatycznie przegrywa. Dodatkowo, wprowadziłam limit liczby rund, który zabezpiecza program przed bardzo długimi lub zapętlonymi rozgrywkami.

W trybie `verbose` po uruchomieniu programu wypisywane są szczegóły rozgrywki, a po każdym ruchu wypisywane są szczegóły tury.

Po zakończeniu rozgrywki funkcja zwraca:

- zwycięzcę (None jeśli remis),
- liczbę wykonanych rund,
- końcowy stan planszy.

4.7. main.py

Moduł `main.py` jest punktem startowym programu i odpowiada za obsługę argumentów przekazywanych z CLI. Do parsowania argumentów wykorzystana została biblioteka `argparse`.

Użytkownik może ustawić wiele parametrów rozgrywki:

- rozmiar planszy,
- maksymalną głębokość przeszukiwania,
- algorytm przeszukiwania,
- limit rund,
- tryb szczegółowego wypisywania ruchów.

Przed uruchomieniem gry wykonywana jest walidacja parametrów. Następnie tworzona jest plansza początkowa i uruchamiana funkcja `play_game()` z modułu `engine.py`.

Moduł odpowiada również za pomiar czasu działania programu oraz wypisywanie końcowych statystyk:

- liczba odwiedzonych węzłów drzewa,
- czas działania algorytmu,
- liczba rozegranych rund,
- zwycięzca gry.

Dzięki wykorzystaniu argumentów CLI program można łatwo uruchamiać z różnymi parametrami testowymi bez potrzeby modyfikacji kodu.

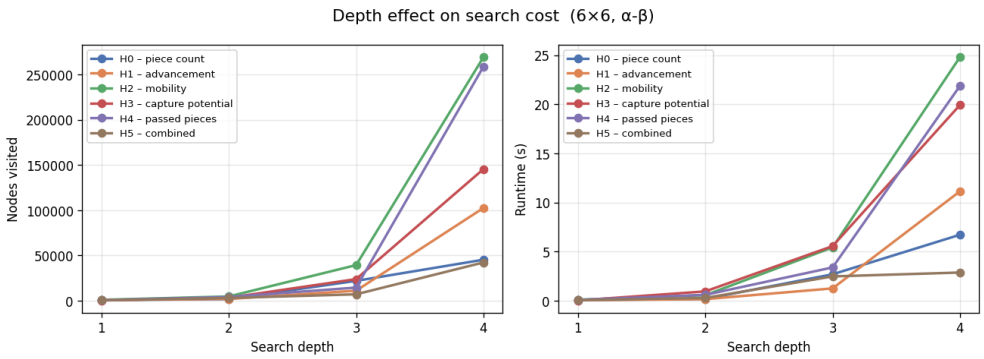
5. Testy i obserwacje

W ramach testów przygotowałam Jupyter Notebook (`analysis.ipynb`), który został wygenerowany przy użyciu generatywnej sztucznej inteligencji (Claude). Służy on do przeprowadzania eksperymentów na różnych konfiguracjach gry Breakthrough oraz porównania wpływu parametrów takich jak: głębokość przeszukiwania, rozmiar planszy oraz użyta heurystyka.

Eksperymenty były uruchamiane automatycznie dla wielu kombinacji ustawień. Następnie wyniki były zbierane i analizowane z użyciem biblioteki `pandas` oraz `matplotlib`.

5.1. Głębokości i heurystyki

Pierwszy eksperyment badał wpływ głębokości przeszukiwania oraz użytej heurystyki na liczbę odwiedzonych węzłów i czas działania algorytmu (plansza 6×6, alfa-beta).

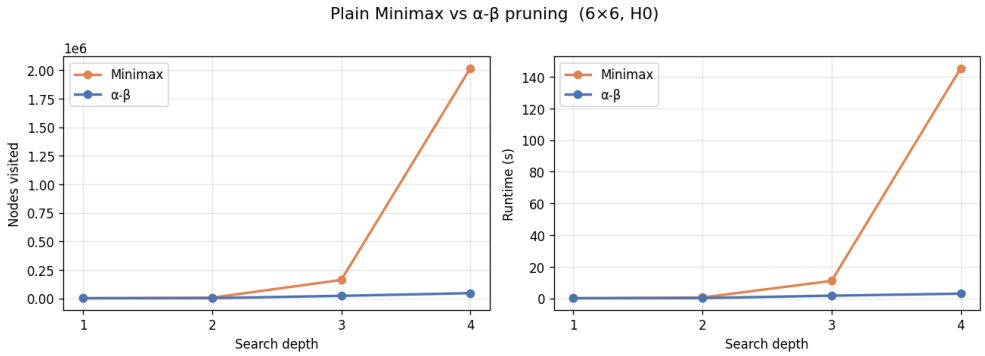


Rysunek 1: Wpływ głębokości na koszt obliczeniowy (6×6, $\alpha - \beta$)

Z powyższych wizualizacji wyraźnie wynika, że wzrost głębokości powoduje wykładniczy wzrost liczby odwiedzonych węzłów oraz czasu działania algorytmu. Dla $depth \leq 3$ czas obliczeń jest znikomy, jednak przy $depth = 4$ zaczyna być już mocno zauważalny. Heurystyki nie zmieniają trendu kosztu, jednak możemy zauważyć, że koszt obliczeniowy mocno zależy od wybranej heurystyki.

5.2. Minimax vs alfa-beta

W tym eksperymencie porównano klasyczny Minimax z wersją z cięciami alfa-beta. Oba algorytmy zwracają te same decyzje, różnią się jedynie kosztem obliczeniowym.

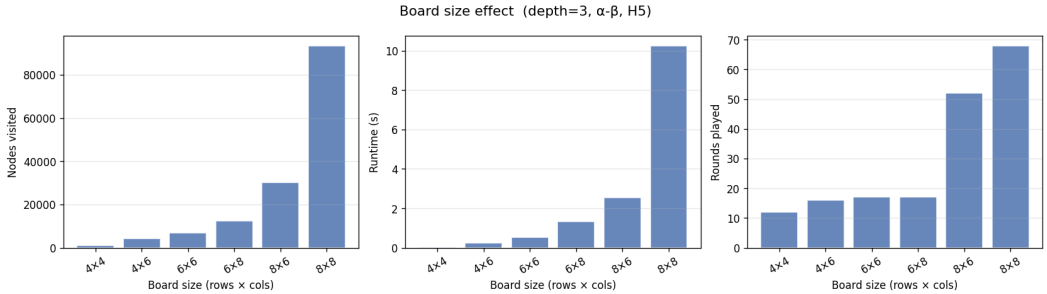


Rysunek 2: Porównanie Minimax i $\alpha - \beta$ (6×6, H0)

Możemy zauważyć, że alfa-beta cięcia znacząco redukują liczbę odwiedzanych węzłów, a tym samym skracają czas obliczeń. Różnica wydajności rośnie wraz z głębokością – dla $depth = 4$ różnica jest już bardzo wyraźna.

5.3. Rozmiar planszy

Ten test badał, jak zmiana rozmiaru planszy wpływa na liczbę odwiedzonych stanów i czas obliczeń (przy stałej głębokości i alfa-beta).

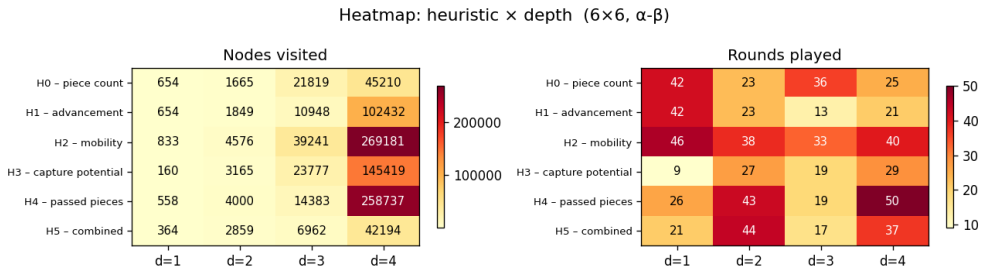


Rysunek 3: Wpływ rozmiaru planszy na liczbę stanów i czas ($\alpha - \beta$)

Tutaj również wyniki są dosyć oczywiste. Liczba odwiedzonych stanów rośnie wykładniczo także przy zwiększaniu rozmiaru planszy.

5.4. Heurystyki

Ostatni eksperyment porównał różne heurystyki przy różnych głębokościach.



Rysunek 4: Porównanie heurystyk (liczba węzłów i długość gry)

Możemy łatwo zauważyć, że różne heurystyki generują różne style gry - niektóre prowadzą do rozgrywek krótszych i bardziej dynamicznych, a inne do dłuższych i mniej dynamicznych. Z powyższej wizualizacji wynika również, że pozornie lepsze, bardziej złożone heurystyki nie zawsze oznaczają lepszą wydajność.

5.5. Wnioski

Na podstawie przeprowadzonych testów można zauważyć kilka rzeczy:

- głębokość przeszukiwania ma największy wpływ na koszt obliczeń,
- alfa-beta cięcia skutkują ogromną poprawą wydajności,
- rozmiar planszy bardzo szybko zwiększa złożoność problemu,
- heurystyki wpływają bardziej na jakość gry niż na sam koszt obliczeń.

6. Podsumowanie

Projekt pozwolił mi przede wszystkim zobaczyć w praktyce, jak wygląda implementacja gry przeciwko komputerowi oraz jak buduje się algorytmy decyzyjne oparte na przeszukiwaniu drzewa gry. Przy okazji nauczyłam się, jak w praktyce działa Minimax i jak bardzo jego wydajność można poprawić dzięki cięciom alfa-beta.

Całość dała mi lepsze zrozumienie tego, jak projektuje się proste AI do gier planszowych i jak ważne są optymalizacje w algorytmach przeszukiwania.

7. Materiały źródłowe

1. Materiały do kursu
2. *Breakthrough (board game)*
3. *Minimax*
4. *Alpha-beta pruning*